

第 109 组《英雄试炼》课程设计作业报告

一、程序功能介绍

1.1 项目概述

本项目名称为《英雄试炼》，是一款使用 Python 和 PyQt5 开发的回合制英雄战斗小游戏。游戏围绕“蘑菇基类派生英雄”的主题展开，玩家需要先在英雄选择界面中把基础蘑菇拖动到舞台上，再通过不同武器图标将其派生为战士、弓箭手或法师，组成三人小队后进入战斗界面，挑战熔岩领主及其召唤的小兵。

项目的核心目标不仅是实现一个可运行的小游戏，也是在程序中体现面向对象程序设计思想。基础蘑菇作为父类，战士、弓箭手和法师作为子类，继承基础属性并扩展不同战斗特性；界面中的角色、舞台、武器、镜子、垃圾桶、敌人、技能动画等对象也分别封装为类，通过拖拽、点击、碰撞检测和信号槽机制完成交互。

1.2 游戏流程

程序启动后首先进入开始界面。开始界面包含游戏标题、开始游戏、帮助和退出按钮，用户点击“开始游戏”后进入英雄选择界面，点击“帮助”可以查看玩法说明，点击“退出”关闭程序。

在英雄选择界面中，玩家需要完成三名英雄的创建。界面左侧提供基础蘑菇，玩家将基础蘑菇拖动到三个舞台位置上，再将剑、弓、法杖三种武器图标拖动到基础蘑菇身上进行派生。派生规则如下：

武器	派生英雄	基础定位
剑	战士	生命值高，单体伤害较强，可为队友提升攻击力
弓	弓箭手	擅长群体攻击，可为队友附加反伤效果
法杖	法师	可攻击敌人，也可治疗队友并降低敌人攻击能力

当三个舞台都放置了已经派生完成的英雄后，玩家可以点击开始战斗按钮进入战斗界面。如果舞台为空，或者仍然存在未派生的基础蘑菇，程序会弹出提示，阻止玩家进入战斗。

战斗界面采用回合制规则。玩家回合中，每名存活英雄每回合只能行动一次。将英雄拖动到敌人身上可以发动攻击，将英雄拖动到队友身上可以释放辅助技能。所有英雄行动完毕后进入敌人回合，敌人会按规则进行普通攻击、群体攻击或召唤小兵。若所有敌人被击败，玩家胜利；若所有英雄阵亡，游戏失败并返回选择界面。

1.3 任务 1 功能实现

任务 1 要求实现右上角显示功能，并展示基础蘑菇、派生英雄、镜子、垃圾桶和武器图标等交互对象的信息。

本项目在英雄选择界面右上角设置了信息显示区域 `info_label`。用户点击角色或部分功能对象时，信息区域会即时更新。基础蘑菇显示其名称、基础攻击力、基础血量、继承关系和派生说

明；战士、弓箭手、法师显示自身攻击力、血量、继承自基础蘑菇的关系以及各自特性。这样玩家可以在选择阶段直观看到不同英雄之间的差异。

基础蘑菇作为父类，主要属性包括名称、攻击力、血量、特性描述和父类信息。战士、弓箭手、法师继承基础蘑菇类，并覆盖攻击力、血量和特性描述。点击派生后的英雄时，程序通过角色类型查找对应数据类，再调用格式化方法生成右上角显示文本。

除英雄信息外，镜子、垃圾桶和武器也提供了交互说明。点击镜子时，右上角会说明镜子可以复制已经派生好的英雄，并提供深拷贝和浅拷贝两种模式；点击垃圾桶时，显示其可以删除已有角色；点击武器图标时，显示武器与英雄派生之间的对应关系。通过这些提示，玩家可以不依赖外部说明直接理解界面操作。

任务 1 中还实现了以下扩展功能：

1. 三个蘑菇英雄类体现继承关系，基础蘑菇为父类，战士、弓箭手、法师为子类。
2. 武器图标可拖动到基础蘑菇身上完成派生，派生后角色图片、类型和属性同步变化。
3. 镜子可复制已经派生完成的角色，并区分深拷贝和浅拷贝。
4. 垃圾桶可删除舞台上的已有角色，浅拷贝组成员会产生联动删除效果。
5. 右上角显示区域会根据当前点击对象动态更新内容。

1.4 任务 2 功能实现

任务 2 主要负责美术细节和操作反馈。项目中的界面资源包括开始界面背景、选择界面背景、战斗背景、游戏标题、按钮图标、英雄贴图、敌人贴图、武器图标、镜子、垃圾桶、舞台、箭矢、火球、闪电球等素材，使游戏从单纯的控件窗口变成完整的图形化小游戏。

在交互反馈方面，程序为可拖动对象设置了手型鼠标指针，为武器按钮设置了悬停和按下时的背景变化效果，使用户能够区分可操作对象和普通背景。拖动英雄或武器时，程序会生成对应的拖拽图标，放置到有效目标上后触发派生、攻击或辅助技能；无效条件下则会弹出提示或忽略操作。

战斗阶段加入了多种动画效果。英雄与敌人入场时会从屏幕外移动到指定位置；战士攻击时会前冲再返回；弓箭手会发射箭矢；法师会释放闪电球；敌人攻击时会释放火球；治疗、攻击提升和反伤效果通过飞行光点、状态标签和浮动文字表现。角色受到攻击或获得效果时会产生抖动动画，生命值通过血条实时显示。

项目还处理了窗口缩放和全屏显示。开始界面、选择界面和战斗界面都在 `resizeEvent` 中根据当前窗口大小重新计算背景、按钮、角色和舞台的位置与尺寸，保证在不同分辨率下界面布局相对稳定。F11 可切换全屏，Esc 可退出全屏，提升了实际运行时的完整度。

二、项目各模块与类设计细节

2.1 总体模块结构

项目主要由三个 Python 文件和一个图片资源目录组成：

文件或目录	作用
`main_window.py`	游戏开始界面，负责启动、帮助、退出和全屏切换

`select_hero.py`	英雄选择界面，负责基础蘑菇放置、武器派生、信息显示、复制和删除
`fight_window1.py`	战斗界面，负责英雄与敌人创建、技能释放、回合控制、动画和胜负判断
`images/`	存放背景、英雄、敌人、按钮和技能动画所需图片资源

三个模块之间的关系较清晰：`main_window.py` 是入口模块，创建开始界面并跳转到选择界面；`select_hero.py` 根据玩家选择生成英雄列表，并把英雄类型、舞台编号、浅拷贝组信息传递给战斗界面；`fight_window1.py` 根据这些数据创建战斗角色，完成回合制战斗逻辑。

2.2 开始界面模块

开始界面主要由 `StartGameBackground` 类实现，该类继承自 `QMainWindow`。它负责加载初始背景和标题图片，并创建开始、帮助、退出三个按钮。

`on_start_clicked` 方法负责界面跳转。点击开始按钮后，程序会记录当前窗口大小和全屏状态，将这些状态同步给英雄选择界面，再显示选择界面并隐藏开始界面。这样可以避免切换窗口时尺寸突变。

`on_help_clicked` 方法使用 `QMessageBox` 显示游戏玩法，包括英雄派生、战斗规则和其他操作。`toggle_fullscreen` 和 `exit_fullscreen` 方法分别负责 `F11` 全屏切换和 `Esc` 退出全屏。

2.3 英雄数据与继承体系

英雄数据定义在 `select_hero.py` 中。`MushroomHero` 是基础蘑菇类，提供名称、攻击力、血量、描述、父类信息和 `format_info` 方法。`Warrior`、`Archer`、`Mage` 继承自 `MushroomHero`，分别表示战士、弓箭手和法师。

该设计体现了继承和多态思想：父类规定英雄数据的共同结构，子类只需要覆盖差异化属性。右上角显示信息时，无论当前角色是基础蘑菇还是派生英雄，都可以通过统一的 `format_info` 方法生成文本，从而减少重复代码。

`HERO_TYPE_TO_CLASS` 字典负责将字符串类型映射到具体类，`get_hero_data_by_type` 函数根据角色类型返回对应英雄对象。这样界面层只需要关心 `hero_type`，不需要直接判断具体类，有利于后续扩展新英雄。

2.4 英雄选择与派生模块

英雄选择界面的主类是 `SelectHeroWindow`。该类负责加载选择背景、基础蘑菇、三个舞台、三种武器、返回按钮、镜子、垃圾桶、开始战斗按钮和信息显示区域。

`DraggableLabel` 用于表示可拖动英雄。它在鼠标按下时记录起始位置，在鼠标移动超过拖拽阈值后创建 `QDrag` 对象并携带英雄类型、舞台编号等数据。当玩家只是点击而不是拖动时，`mouseReleaseEvent` 会调用主窗口的 `show_hero_info`，在右上角显示当前角色信息。

`StageLabel` 表示舞台。它只接受基础蘑菇拖入，并且每个舞台最多放置一个角色。放置成功后调用 `create_hero_on_stage` 创建舞台上的英雄实例。

`DraggableEquipButton` 表示武器图标。剑、弓、法杖分别携带不同 `equip_type`，拖动到基础蘑菇身上后触发派生逻辑。派生时程序会修改角色图片、角色类型、英雄数据对象和位置，使其从基础蘑菇变为对应英雄。

MirrorLabel 表示镜子。它只接受已经派生的英雄，拖入后调用 `try_copy_hero`，弹出 CopyDialog 让用户选择深拷贝或浅拷贝。深拷贝生成完全独立的英雄，浅拷贝则把原英雄和复制英雄加入同一个共享组。

BinLabel 表示垃圾桶。拖入舞台上的英雄后调用 `delete_hero_with_chain` 删除角色。如果被删除角色属于浅拷贝组，则同组角色会一起删除，体现浅拷贝共享资源的含义。

2.5 战斗模块

战斗模块由 `fight_window1.py` 实现，主要类包括 `fighting_hero`、`Enemy`、`Fight_Window`、`arrow`、`Fireball`、`LightningBall` 和 `SupportOrb`。

`fighting_hero` 表示战斗中的我方英雄。它保存英雄类型、攻击力、生命值、是否存活、是否本回合可行动、攻击倍率、反伤值和浅拷贝组编号。它支持拖拽操作：拖动到敌人身上表示攻击，拖动到队友身上表示释放辅助技能。队友接收拖拽后，会根据施法者类型产生不同效果：法师治疗，战士提升攻击倍率，弓箭手附加反伤。

`Enemy` 表示敌方单位，包括熔岩领主和熔岩小兵。敌人也有生命值、攻击力、状态图标和行动逻辑。熔岩领主可以普通攻击、准备群体攻击或准备召唤小兵；小兵主要进行单体攻击。敌人受到英雄拖拽攻击时，会根据攻击者类型计算伤害和动画。

`Fight_Window` 是战斗窗口类，负责读取选择界面传来的英雄列表，创建我方英雄和敌人，安排站位，控制入场动画，维护回合状态，并判断胜负。它通过 `moveable_hero` 记录每名英雄本回合是否还能行动。当所有可行动英雄都行动完毕后，切换到敌人回合；敌人回合结束后重新刷新英雄行动状态，回到玩家回合。

`arrow`、`Fireball`、`LightningBall` 和 `SupportOrb` 是视觉效果类，分别用于弓箭、敌方火球、法师闪电球和辅助技能光点。它们通过 `QPropertyAnimation` 控制位置变化，使技能释放过程更加直观。

2.6 数据传递与状态管理

从选择界面进入战斗时，`on_start_battle` 会遍历三个舞台，生成 `hero_list1`，其中包含英雄类型、舞台编号和浅拷贝组编号。同时，程序还会生成 `shallow_group_list1`，记录浅拷贝组与舞台编号之间的关系。这些数据传递给 `Fight_Window` 后，战斗界面即可复原玩家选择的队伍。

战斗结束后，`_end_battle` 会清空全局列表，重置选择界面的战斗状态，并返回选择界面。这样玩家可以重新组队开始下一局，避免上一局残留的角色、敌人或行动状态影响新游戏。

三、小组成员分工情况

本项目由三名成员协作完成。项目开始阶段，三名成员共同讨论了游戏主题、基本玩法、界面构成和任务拆分方式，确定以“基础蘑菇通过武器派生英雄，并组成队伍挑战敌人”为核心创意。

李锦城主要负责项目的初步实现。他完成了项目基础框架搭建，包括 PyQt5 窗口结构、主要界面雏形、资源加载方式和基本交互流程，为后续任务 1 和任务 2 的开发提供了基础。

于泽凯主要负责任务 1。他重点实现了右上角信息显示、基础蘑菇与派生英雄的数据类设计、点击角色显示属性、武器拖拽派生、镜子复制和垃圾桶删除等功能，使项目能够体现面向对

象中的继承、派生、深拷贝和浅拷贝等知识点。

张家玮主要负责任务 2 以及最终整合。他重点完善了美术细节、图片资源接入、按钮和可拖拽对象的交互反馈、战斗阶段的攻击动画、技能特效、血条显示、状态标签、窗口缩放适配和最终代码整合，保证项目整体能够连贯运行。

四、项目总结与反思

通过本次课程设计，我们完成了一个具有完整流程的 PyQt5 回合制战斗小游戏。项目包含开始界面、英雄选择界面和战斗界面，支持拖拽、点击、派生、复制、删除、战斗、技能、动画、血条、回合切换和胜负判断等功能。相比单个知识点练习，本项目更接近一个小型完整程序，需要同时考虑界面显示、对象设计、状态传递和用户交互。

在程序设计方面，本项目较好地使用了面向对象思想。英雄数据通过父类和子类组织，界面对象通过不同控件类封装，战斗单位和技能动画也分别定义为独立类。这样可以让不同功能分布在相对清晰的模块中，降低了所有逻辑写在一个函数里的混乱程度。

在实现过程中，我们也遇到了一些问题。第一，拖拽交互需要同时处理鼠标事件、拖拽数据、目标接收条件和放置后的状态更新，调试难度比普通按钮点击更高。第二，窗口缩放后角色、舞台、背景和按钮的位置都需要重新计算，否则界面容易错位。第三，战斗回合中既有动画又有状态变化，如果动画尚未结束就切换回合，可能导致逻辑顺序混乱，因此需要用定时器和回调来控制流程。第四，浅拷贝和深拷贝的表现需要在界面和战斗中保持一致，否则玩家难以理解共享资源和独立资源的区别。

本项目仍有可以改进的地方。例如，可以进一步统一中文编码和命名规范，使代码注释与运行提示更清晰；可以增加更多敌人种类和关卡选择，让游戏内容更丰富；可以加入音效和更完整的胜利、失败界面；也可以把全局变量进一步封装到类中，减少模块之间的耦合。

总体而言，本次大作业让我们综合运用了 Python、PyQt5、面向对象、事件处理、拖拽机制、动画和团队协作等内容。通过从想法讨论到功能实现，再到整合测试和报告整理，我们对图形化程序开发流程有了更完整的认识，也加深了对类、对象、继承、派生、拷贝和状态管理等课程知识的理解。